

CRM Hospital São Rafael

Sprint 4 — Exposição das Regras de Negócio em JSON e Views

Integrante	RM
Samuel Baecker	RM98765

Descrição da Solução

O **CRM Hospital São Rafael** é uma aplicação back-end Java/Spring Boot 3 que digitaliza todo o ciclo de aquisição e relacionamento com pacientes do hospital. A solução foi modelada seguindo **Domain-Driven Design**, dividindo o código em pacotes por domínio (lead, agendamento, interação, usuário, notificação e inteligência artificial). Cada domínio expõe dois conjuntos de endpoints: um **JSONController** (`@RestController`) que serve a API REST consumida pelo front-end React/Vite e por integrações externas, e um **ViewController** (`@Controller`) que renderiza páginas HTML server-side com **Thymeleaf**. As páginas Thymeleaf funcionam como **documentação viva** da API: cada uma contém um bloco listando os endpoints relacionados, verbos HTTP e códigos de retorno esperados.

As principais funcionalidades incluem: cadastro de leads com cálculo automático de *lead score* e prioridade; gestão de agendamentos com validação de conflito de horário e data passada; registro de interações (ligações, e-mail, WhatsApp) com detecção semântica de urgência via IA (Spring AI + Anthropic Claude); notificações em tempo real via WebSocket; relatórios diários gerados automaticamente por IA; autenticação JWT com Spring Security; e tratamento centralizado de exceções por meio de *GlobalExceptionHandler* com `@RestControllerAdvice`. A arquitetura segue rigorosamente o padrão em camadas (Controller → Service → Repository), utiliza DTOs com Bean Validation, Mappers dedicados, conversores JPA para enums e documentação OpenAPI/Swagger acessível em `/swagger-ui.html`.

Atendimento aos critérios da Sprint 4

Critério	Como foi atendido
Endpoints separados por domínio (JSON e View) e cada um com documentação acessível	JSONController (@RestController) e ViewController (@Controller) com Swagger UI e Thymeleaf para documentação
Nomenclatura de endpoints, códigos de status HTTP e mensagens de erro padronizadas	RESTful, GET, POST, PUT, PATCH, DELETE 200 OK, 201 Created, 204 No Content, 404 Not Found, 500 Internal Server Error
Funcionalidades propostas, entregues e operando	Lead, agendamento, interação, usuário, notificação, IA
Boas práticas (nomenclatura de pacotes, classes, métodos, DTOs, Mappers, etc.)	Domain-Driven Design, DTOs, Mappers, Spring Security, Spring AI, Spring Data JPA, Spring Websocket, Spring Cloud

Páginas que compõem a solução

URL	Template Thymeleaf	Descrição
/painel	templates/painel.html	Dashboard executivo com KPIs e leads urgentes
/leads	templates/leads/lista.html	Lista paginada de leads + cadastro + documentação
/leads/{id}	templates/leads/detalhe.html	Detalhe do lead com histórico e anexos
/agendamentos	templates/agendamentos/lista.html	Gestão de agendamentos por período
/interacoes	templates/interacoes/lista.html	Histórico de interações e análise
/notificacoes	templates/notificacoes/lista.html	Caixa de notificações do operador
/usuarios	templates/usuarios/lista.html	Gestão de usuários e perfis
/ia	templates/ia/painel.html	Painel de IA: análise de leads e relatório diário
/swagger-ui.html	(gerado por springdoc)	Documentação OpenAPI completa

Estrutura técnica do projeto

Stack: Java 17, Spring Boot 3, Spring Data JPA (Oracle), Spring Security + JWT, Spring AI (Anthropic Claude), Thymeleaf, Bootstrap 5, WebSocket (STOMP), Lombok, MapStruct (mappers manuais), JUnit 5 + Mockito, springdoc-openapi.

Pacotes principais: *controller/* (REST JSON), *view/* (Thymeleaf), *service/*, *repository/*, *model/*, *dto/*, *mapper/*, *exception/*, *config/*, *security/*, *ai/*.

Repositório: <https://github.com/sbaecker2005/CH-java>

Observação sobre prints

Os prints das páginas que compõem a solução devem ser anexados ao final deste documento ou capturados após subir a aplicação (`./mvnw spring-boot:run` dentro de `crm-hospital/`) e acessar as URLs listadas na tabela acima.